



ECOLE
POLYTECHNIQUE
DE BRUXELLES

2025–2026

Computing Project

PIG: GPU Redesign of a Plenoptic Imaging Pipeline and Extension to Multi-View Reconstruction

Niccolò Benedetto
Enrollment number 626872

Prof. Daniele Bonatto
Ferreira Ribeiro Brenno

June 2026



Contents

1	Background Context and Motivation	2
1.1	Integral Imaging Fundamentals	2
1.2	Existing Prototype	3
1.3	Project Objectives	4
1.4	Computational Resources	5
2	GPU Pipeline Redesign	7
2.1	GPU Point Cloud Generation	7
2.2	GPU Post-Processing	8
2.3	Experimental Validations and Results	9
3	GPU-Based Multi-View Pipeline	13
3.1	Multi-View Dataset	14
3.2	GPU Multi-View Point Cloud Generation	14
3.3	GPU Multi-View Registration	14
3.4	Experimental Validations and Results	16
4	Conclusion	19

Abstract

This report presents the redesign of an existing CPU plenoptic image generation (PIG) pipeline toward a GPU implementation using CUDA. This work focuses on accelerating the most computationally expensive stages of the existing pipeline. In addition, the new GPU pipeline is extended to support multi-view datasets (RGB + depth) by means of GPU-based point cloud registration and merging. Experimental results demonstrate significant runtime improvements and validate the geometric consistency and efficiency of the proposed multi-view pipeline regarding the single-view disocclusion problems.

Acknowledgements

I would like to thank Brenno Ribeiro Ferreira and prof. Daniele Bonatto for their supervision and guidance throughout this project. Their support and feedback were fundamental to the development of this work.

Background Context and Motivation

Three dimensional display technologies aim to reproduce depth perception and spatial visualization in a better way than conventional two-dimensional displays [Gen13]. Traditional stereoscopic systems generate depth perception by presenting different images to each eye; however they suffer from the accommodation-convergence conflict which may produce visual fatigue during prolonged observation [Hof+08].

Among different approaches proposed for glass-free 3D visualization, Integral Imaging (InI) represents a promising solution due to its capability to reconstruct light fields through microlens arrays without wearable devices [MJ18]. The concept was originally introduced by Lippmann in 1908, where multiple perspectives of the same scene were captured and reconstructed using an array of microlenses [Lip08].

Today, modern integral imaging systems combine microlens arrays and computational rendering algorithms to generate plenoptic images suitable for three-dimensional visualization [MJ18]. In this context the Plenoptic Image Generator (PIG) pipeline [Fer+26] converts RGB+Depth (noted as RGB+D in this work) data into plenoptic images for integral imaging displays.

1.1 Integral Imaging Fundamentals

Integral Imaging (InI) is a glasses-free three-dimensional visualization technique based on light-field reconstruction through a Microlens Array (MLA). A plenoptic image, composed of a grid of microimages, is displayed behind the MLA, allowing each microlens to project a slightly different view of the scene toward the observer. This enables depth perception and motion parallax without requiring wearable devices. [Fer+26]

The display geometry is mainly defined by the MLA pitch D , focal length f , and the gap B between the MLA and the display panel. These parameters determine the reconstruction properties of the system. The following figure (adapted from the figure 2 in [Fer+26]) explains better the setup of the Integral Imaging.

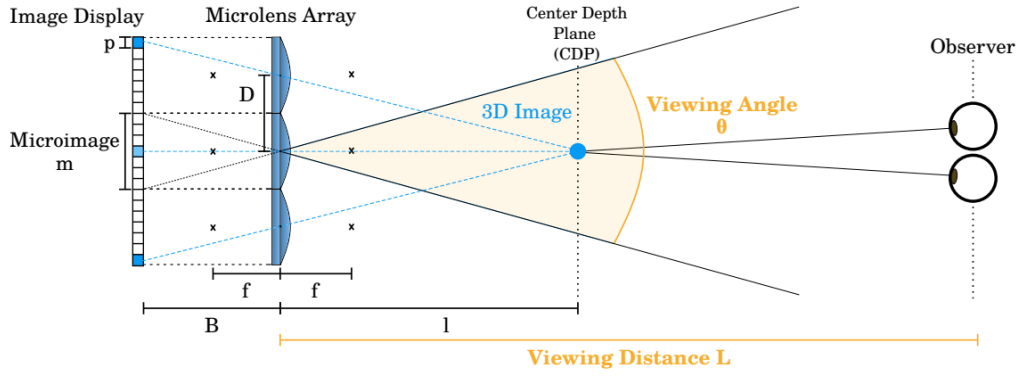


Figure 1.1: Geometric model of an Integral Imaging display system. Adapted from [Fer+26].

The reconstructed image is focused around the Center Depth Plane (CDP), whose position is derived from the thin lens equation:

$$l = \frac{Bf}{B - f} \quad (1.1)$$

The existing prototype adopted uses a square MLA configuration with a pitch of 1.6056 mm, focal length of 8.028 mm, and a display–MLA gap configured for real-mode operation.

1.2 Existing Prototype

The Plenoptic Image Generator (PIG) converts an RGB image and its depth map, where each pixel corresponds to a depth for that point (“RGB+D” data notated in this work), into a plenoptic image suitable for integral imaging displays (pipeline presented in [Fer+26]). A plenoptic image is a 2D image composed by many microimages, each of these associated to a microlens. During optical reconstruction, each microlens projects its microimage into space from a slightly different viewpoint, enabling the reconstruction of the light field and 3D perception for the observer.

The original PIG pipeline is organized into four main stages: Pre-Processing, Point Cloud Reconstruction, Plenoptic Rendering, and Post-Processing, as illustrated in 1.2. The current pipeline is CPU-based, except for the Plenoptic Rendering stage.

The Pre-Processing stage applies super-resolution upscaling to the RGB image and the depth map in order to densify the reconstructed point cloud. This operation reduces crack artifacts caused by the sparse nature of single-view point-cloud reconstruction. In fact, if only a single view point is available, the sampling of the scene could be discontinuous, especially in regions with large depth variations. As a consequence, adjacent pixels in the original image may project to spatially distant positions in 3D space, leaving empty regions and holes [Feh04; Mud+12].

The Point Cloud Reconstruction stage converts RGD+D data into a 3D point cloud by taking into account the intrinsic camera parameters. Assuming a pinhole camera model, each pixel (x_i, y_i) associated with depth Z_i is backprojected into 3D space according to:

$$(X_i, Y_i, Z_i)^T = \left(\frac{(x_i - p_x)Z_i}{f_c}, \frac{(y_i - p_y)Z_i}{f_c}, Z_i \right)^T \quad (1.2)$$

where f_c denotes the camera focal length and (p_x, p_y) the principal point coordinates.

The plenoptic stage computes each microimage by projecting all point through each microlens capable of observing them, using a pinhole camera model (1.2). However when a point is located close to the MLA plane ($Z_i \rightarrow 0$) the projection

$$\frac{B}{Z_i}$$

diverges, becoming numerically unstable. The original implementation, presented in [Fer+26], addresses this issue by filtering points below a predefined depth threshold before projection.

Finally, the Post-Processing stage reduces the remaining crack artifacts generated during the rendering. In fact, despite super-resolution, some pixels remain empty due to the discrete nature of the point cloud. Invalid pixels are identified through the alpha channel and reconstructed through local median filtering applied independently inside each microimage region. Local median filtering is effective in this context since typically crack pixels are local outliers and such filter reconstructs the missing value by preserving edges.

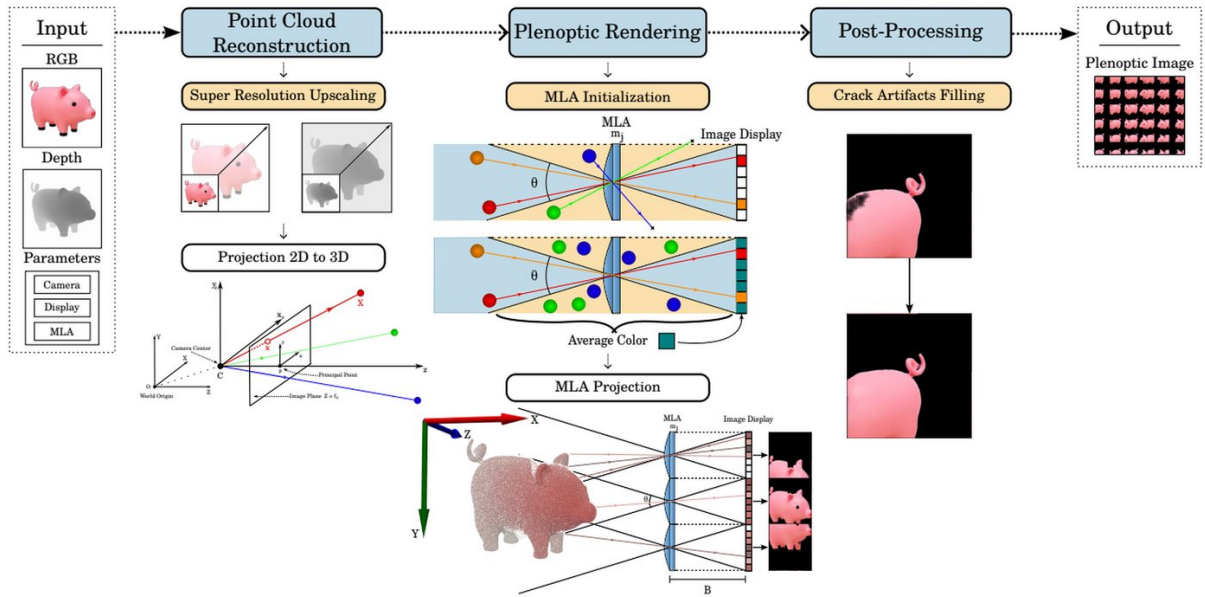


Figure 1.2: Overview of the original PIG rendering pipeline. Adapted from [Fer+26].

1.3 Project Objectives

The existing CPU-based pipeline works but suffers from runtime performance, due to its sequential nature. Stages like Point Cloud Reconstruction and Post Processing involve large amount of pixel-wise and point-wise operations making them computationally expensive. Moreover the

resulting plenoptic image, obtained by Ferreira et al., depicts disocclusions caused by hidden areas that become visible in the rendering process.

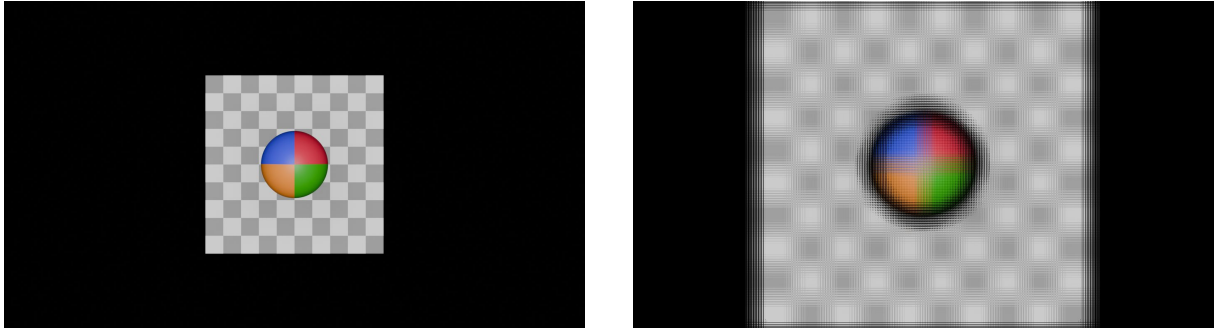


Figure 1.3: Example of crack and disocclusion artifacts generated by single-view point-cloud rendering. On the left the initial RGB image from the *ball* dataset; on the right the plenoptic result through the existing pipeline. Crack artifacts appear as small black gaps and discontinuities between neighboring projected samples, particularly around the sphere boundaries and background pattern, while disocclusions correspond to new regions revealed near object contours by changing microlens viewpoint during the rendering. Adapted from [Fer+26].

These limitations motivate naturally objectives of this work. The first one is to redesign the most time computationally expensive stages of the original pipeline into a GPU execution while preserving rendering correctness and visual consistency. The high level of computation independence here is the key for a CUDA programming approach. The second objective is to extend the new pipeline by implementing a new multi-view reconstruction pipeline by taking into account datasets with multiple RGB+D data in order to address the disocclusions problem.

1.4 Computational Resources

All the experiments were conducted on the remote machine of the LISA laboratory. The computational resources available are listed in the table below.

Table 1.1: GPU specifications

Device name	NVIDIA GeForce GTX 1080 Ti
Compute capability	6.1
Max Threads per block	1024
Max Threads per Multiprocessor	2048
Shared Memory per block	48 KB
Shared Memory per Multiprocessor	96 KB
Registers per Block/Multiprocessor	65536
Max Grid Size	[2147483647 65535 65535]
Max Block Dimensions	[1024 1024 64]
Single precision FLOP/s (FP32)	3914.99 GFLOP/s
Double precision FLOP/s (FP64)	1957.49 GFLOP/s
#Multiprocessor (SM)	28
Processor Clock Rate	1.607 GHz
Threads per Warp	32
Global Memory Bandwidth	484.44 GB/s
Global Memory Size	10.9999 GB
Constant Memory Size	64 KB
L2 Cache Size	2816 KB
Memory Clock Rate	5505 MHz

GPU Pipeline Redesign

The original PIG prototype already exploits GPU acceleration during the plenoptic rendering stage, while the remaining stages were entirely executed on CPU. The main computational bottlenecks were concentrated in the post-processing and point cloud generation stages. On the other hand the pre-processing stage is already enough fast in CPU, therefore the pipeline was redesigned to bring the remaining two most expensive stage on GPU.

Table 2.1: CPU pipeline execution times.

Stage	Time (ms)	Percentage
Pre-Processing	65.20	3.81%
Point Cloud Generation	255.81	14.93%
Plenoptic Rendering	548.34	32.00%
Post-Processing	844.01	49.26%
Total	1713.36	100%

The old pipeline has not been discarded but it has been introduced a boolean configuration flag in order to switch dinamically between these two and compare effectively the benchmarks.

2.1 GPU Point Cloud Generation

In the point cloud generation stage every RGB+D pixel must be sequentially projected into 3D space one at time. For large datasets, this means millions of point-wise operations. Since each pixel can be processed independently, the stage is naturally suitable for GPU parallelization. The redesign strategy consisted in migrating the projection and filtering stages to CUDA kernels while preserving the original CPU logic.

The original CPU implementation consisted of three sequential steps: point cloud initialization $\rightarrow$ 2D-to-3D projection $\rightarrow$ point cloud adjustment to display.

Among these stages, only the last two were migrated to GPU. Point cloud initialization was intentionally kept on CPU because benchmarks showed that its execution time was already way to negligible compared to the remaining stages (see [2.1](#)).

The projection applies for each pixel the inverse pinhole camera model (1.2) to create a 3D point, part of the point cloud. The GPU programming choices establish two kernels, one to handle the projection and another to handle pixel validity. In the first kernel, invalid pixels where depth is close to 0 or at the background value are skipped. The CPU point cloud generation stage addresses the filtering by means in-place compaction ables to maintain the final data arrays without holes: scan every point of the cloud, if invalid skip it, otherwise keep it and increment a counter. Such approach would yield race conditions considering millions of threads in GPU trying to access the same pointer/element. Therefore the new dedicated validity kernel works in a different way. Each CUDA thread processes one depth sample independently and generates a binary validity mask:

$$mask_i = \begin{cases} 1 & \text{if } Z_i < Z_{bg} \\ 0 & \text{otherwise} \end{cases} \quad (2.1)$$

where Z_{bg} represents the background depth threshold.

After masking, valid points become sparse inside the original image data arrays. For faster memory access during GPU processing, the data needs to be stored in a compact contiguous way. CUDA already provides built-in method to solve this problem efficiently. In particular, by exploiting NVIDIA CUB primitives, we have access to a method `cub::DeviceScan::InclusiveSum()` capable to optimize prefix-sum operations, eliminating the need for atomic operations. Given a binary validity mask:

$$[1, 0, 1, 1, 0, 1] \quad (2.2)$$

the inclusive scan produces:

$$[1, 1, 2, 3, 3, 4] \quad (2.3)$$

where each value represents the compact destination index of a valid point inside the output point cloud.

Finally, after stream compaction, valid pixels are reprojected into 3D coordinates and scattered directly into compact output buffers through the second kernel, which simply applies final geometric normalization and positioning to the point cloud by launching one thread per valid point.

2.2 GPU Post-Processing

The post-processing, responsible for correcting visual artifacts generated during plenoptic image rendering, is the dominant computational bottleneck due to many filtering operations applied over the plenoptic images.

The GPU redesign followed the CPU logic by focusing on two main operations: crack artifact filtering and microimage rotation.

In the crack filtering stage, we proposed to apply the median filter only after decomposing the plenoptic image into independent microimage Regions of Interest (ROIs), where each ROI corresponds to a single MLA lenslet region.

From a CUDA perspective, this approach should improve memory locality by forcing neighborhood accesses inside compact spatial regions, reducing the traffic from global memory during filtering.

Therefore, the filtering kernel was designed for a parallel computation: each CUDA thread processes one pixel and the filtering is applied only to crack values. This thread independence avoids synchronization operations.

At this point the problem was how to properly map the CUDA threads to the exact pixel within the ROI. The problem was decomposed as follows:

- `grid.x` and `grid.y` represent local pixel coordinates inside a microimage ROI,
- `grid.z` identifies the processed microimage region.

This organization naturally reflects the structure of the plenoptic image:

$$thread(x, y, z) \rightarrow pixel(x, y) \text{ inside microimage } z \quad (2.4)$$

We need to take into account that CUDA limits the maximum depth of `grid.z` to 65535 blocks. Since the total number of microimages may exceed this hardware constraint, the implementation introduces a batching strategy. The complete set of microimages is partitioned into multiple subsets, where each subset is processed through an independent kernel launch.

Finally, the final step rotates each microimage by 180 degrees in order to match the physical optical configuration of the display system.

The rotation kernel follows a classic thread-pixel formulation where each CUDA thread: computes local microimage coordinates $\rightarrow$ evaluates the rotated local position $\rightarrow$ remaps the corresponding output pixel. The kernel operates directly in local microimage coordinates.

2.3 Experimental Validations and Results

The GPU redesign was experimentally evaluated by considering both rendering consistency and computational performance with respect to the original CPU pipeline, which plenoptic output is considered as ground truth and time performance as baseline for benchmarking. Rendering consistency was quantitatively evaluated using the Peak Signal-to-Noise Ratio (PSNR) and the experiment was executed on the dataset `ball.json` (1.3). Finally, CUDA profiling by using NVIDIA Profiler was executed in order to observe the behaviour of the kernels.

The point cloud generation and post processing stages were validated independently. The GPU post processing achieved:

$$PSNR_{PP} \approx 33.9 \text{ dB} \quad (2.5)$$

We could not improve further this result most likely due to the neighborhood ordering taken into account for the filtering or the crack-mask application that the GPU redesign introduced respect to the CPU implementation.

After that the point cloud generation GPU was validated. Disabling the post processing stage from both CPU and the GPU pipeline and comparing the results the PSNR dropped:

$$PSNR_{PCG} \approx 31 \text{ dB} \quad (2.6)$$

Since the post-processing stage independently achieved higher PSNR values when isolated, this suggests that the dominant rendering discrepancy originates from the GPU point cloud generation stage itself. The most likely source of discrepancy is the ordering of the point projected during the pinhole projection since the CPU version uses a sequential ordering but GPU does not. Refinements like evaluating float versus double precision during the computation or the stream compaction with NVIDIA CUB were explored, in order to reproduce as closely as possible the PSNR to ∞ without any improvements.

Table 2.2 summarizes the execution times measured for both the original CPU pipeline and the redesigned GPU pipeline.

Table 2.2: CPU vs GPU pipeline execution times.

Stage	CPU (ms)	GPU (ms)
Pre-Processing	65.20	26.76
Point Cloud Generation	255.81	128.11
Plenoptic Rendering	548.34	36.45
Post-Processing	844.01	101.69
Total	1713.36	293.01

The redesigned GPU pipeline reduced the total execution time from approximately:

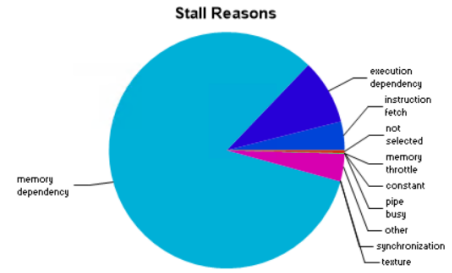
$$1713 \text{ ms} \rightarrow 293 \text{ ms} \quad (2.7)$$

corresponding to an overall speedup of approximately:

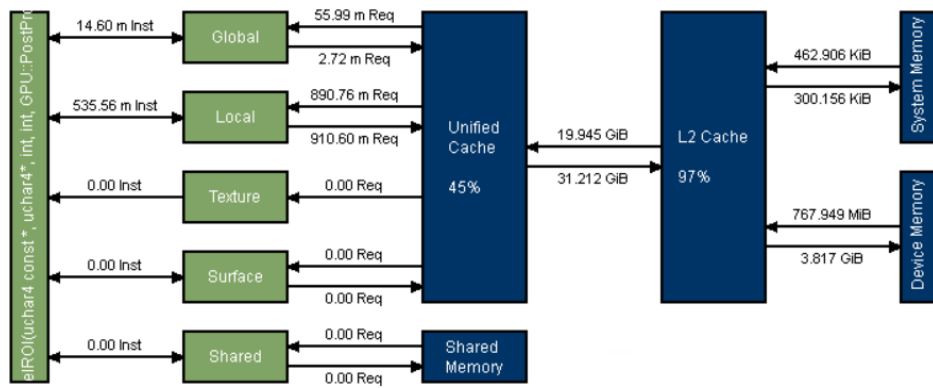
$$\frac{1628}{293} \approx 5.8\times \quad (2.8)$$

Variable	Achieved	Theoretical	Device Limit	Grid Size: [1,1,65535] (65535 blocks)Block Size: [16,16,1] (256 threads)
Occupancy Per SM				
Active Blocks		8	32	
Active Warps	56.7	64	64	
Active Threads		2048	2048	
Occupancy	88.6%	100%	100%	
Warp				
Threads/Block		256	1024	
Warps/Block		8	32	
Block Limit		8	32	
Registers				
Registers/Thread		32	255	
Registers/Block		8192	65536	
Block Limit		8	32	
Shared Memory				
Shared Memory/Block		0	98304	
Block Limit		0	32	

(a) Kernel latency



(b) Stalls analysis



(c) Memory Analysis

Figure 2.2: Crack Kernel Profiling.

For the second kernel profiling shows an high occupancy of 88.6% suggesting that the introduced parallelism seems work well. Warp analysis reveals that memory dependency is by far the dominant stall source, highlighting that the execution waits for data fetched from the memory. In addition, the memory diagram indicates an intensive cache transfer, with the L2 cache that achieves 97% utilization and several gigabytes of data moved between cache levels and device memory. Compared with to the previous kernel, we are closer to a bandwidth memory saturation. This behaviour is consistent with a crack evaluation stage where per-pixel computations and neighboring accesses image samples are performed, followed by an update of the output structures.

GPU-Based Multi-View Pipeline

Since a single RGB+D acquisition only observes the scene from one viewpoint, regions hidden behind foreground cannot be reconstructed correctly. In the ball dataset scenario, this disocclusion produces dark boundaries around the sphere in the final plenoptic rendering. That motivates a new setup composed of three calibrated cameras observing the same scene from different viewpoints in order to compensate occluded regions in the reference camera.

The ball multi-view dataset used for the testing the pipeline extends the single-view reference setup by adding the RGD+D data from two additional cameras that observe the scene as shown in the following figure:

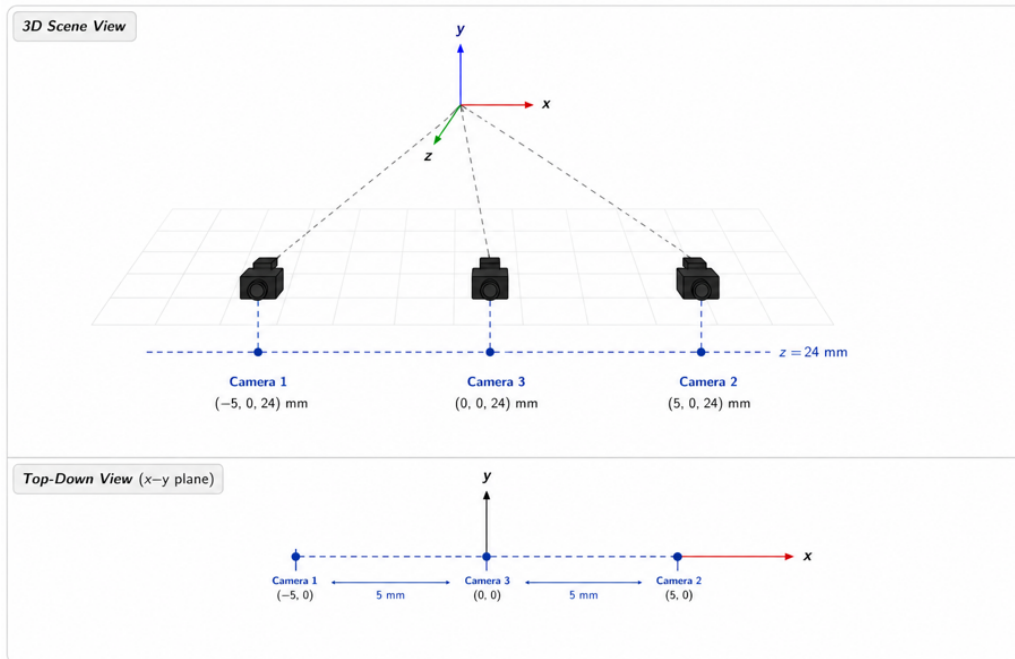


Figure 3.1: Multi-View Camera Setup.

The left and right cameras observe complementary regions of the scene, allowing the reconstruction pipeline to recover geometry that is not visible from the central viewpoint alone.

The proposed extension introduces a new GPU-based multi-view pipeline capable of generating one point cloud per camera (stage 1), transforming all point clouds into a common world

reference frame and merging all into a unified cloud (stage 2), rendering the final plenoptic image (stage 3), post processing (stage 4).

3.1 Multi-View Dataset

The calibration information of the new multi-view dataset (RGB images, depths, intrinsic and extrinsic) is loaded from a JSON file through a dedicated `CameraCalibration` module. Each camera stores:

- focal length in pixels,
- principal point,
- camera center position in world coordinates,
- Euler rotation angles.

The dataset loader initializes one point cloud data container per camera and prepares the parameters required by the GPU reconstruction stages.

3.2 GPU Multi-View Point Cloud Generation

The first stage of the multi-view pipeline generates one point cloud for each camera independently. The implementation reuses the GPU-based `project2Dto3D()` function previously introduced in the single-view pipeline, that takes into account the pinhole projection [1.2](#).

At this step, the generated points are still expressed in the local camera coordinate system:

$$p_{camera} = (X, Y, Z)$$

where the origin corresponds to the optical center of the current camera.

3.3 GPU Multi-View Registration

All point clouds, expressed in their own local camera reference, must be transformed into a common world coordinate system in order to combine them into a single coherent and complete point cloud of the scene.

This transformation is performed by the `MultiViewRegistration` stage where two kernels are used: one for merging the point clouds into the central camera's coordinate system and one for detecting and removing possible duplicate pixels.

The first kernel applies the rigid body transformation to all points. We know that camera extrinsics are commonly expressed using the world-to-camera formulation ([\[Ziv26\]](#)):

$$p_{camera} = R p_{world} + t \tag{3.1}$$

where R is the camera rotation matrix, t is the translation vector, p_{world} is a point expressed in world coordinates and p_{camera} is the same point expressed in camera coordinates.

Since the reconstructed point clouds are expressed in camera coordinates, the inverse transformation must be applied in order to recover world-space geometry. In addition the given calibration information used, directly stores the camera center position C in world coordinates rather than the classical extrinsic translation vector $t = -RC$. Mathematically, starting from the previous equation:

$$p_{camera} - t = Rp_{world}$$

$$R^{-1}(p_{camera} - t) = p_{world}$$

$$p_{world} = R^{-1}p_{camera} - R^{-1}t \quad (3.2)$$

and recalling that $t = -RC$ the final transformation becomes

$$p_{world} = R^{-1}p_{camera} + C \quad (3.3)$$

where R^{-1} aligns the local camera coordinate system with the global world reference frame and C translates the point cloud into its correct world position. The rotation matrix is orthogonal, which simplifies the inverse computation R^{-1} by taking the transpose [Kle01]. Here the first kernel uses classic CUDA thread linearization, assign one thread per point. This simple mapping is well suited to CUDA and avoids synchronization overhead. In other words, this is a good problem to CUDA because it is optimized for it.

performs pretty well since we implemented the classic CUDA thread linearization where we associate one thread per point, by avoiding any kind of synchronization.

After registration, all transformed point clouds are concatenated into a unified structure, by allocating contiguous arrays for X coordinates, Y coordinates, Z coordinates, RGB colors.

The three point clouds are merged sequentially:

$$Cloud_1 + Cloud_2 + Cloud_3$$

producing a single global point cloud suitable for plenoptic rendering.

The second kernel was also introduced in order to remove duplicate: possible points belonging to different point clouds that, after the rigid body transformation, occupy the same spatial region within a small distance tolerance. The current implementation allocates a GPU validity mask and prepares the kernel for spatial duplicate detection based on a tolerance threshold. However, the duplicate filtering logic is currently left as future work and is not yet fully enabled in the final pipeline.

3.4 Experimental Validations and Results

The multi-view pipeline was experimentally evaluated from different perspectives considering the ball dataset. First, execution times were measured in order to observe the performance behaviour of the new introduced stages. Second, the visual quality of the generated plenoptic image was analyzed to verify whether the multi-view reconstruction effectively solve the disocclusions problem observed in the single-view pipeline. Finally, quantitative metrics based on recovered pixels and plenoptic image density were introduced to measure the amount of structure recovered by the additional viewpoints. CUDA profiling by using NVIDIA Profiler was also performed to investigate the computational behavior of the registration kernel.

Table 3.1 reports the performance results:

Table 3.1: GPU Multi-View Pipeline Execution Times

Stage	Time [ms]	Percentage
MultiView Point Cloud GPU	605.83	73.41%
MultiView Registration GPU	46.38	5.62%
Plenoptic Rendering	61.02	7.39%
Post Processing GPU	112.07	13.58%
Total Execution Time	825.31	100%

We can suppose that this new pipeline will increment the total average time of the benchmark compared to the GPU single-view, since it takes into account multiple point clouds. Therefore, rather than the time performance optimization the main focus of the multi-view pipeline was the correctness about the reconstructing geometry.

This motivates a qualitative analysis of the plenoptic image result.

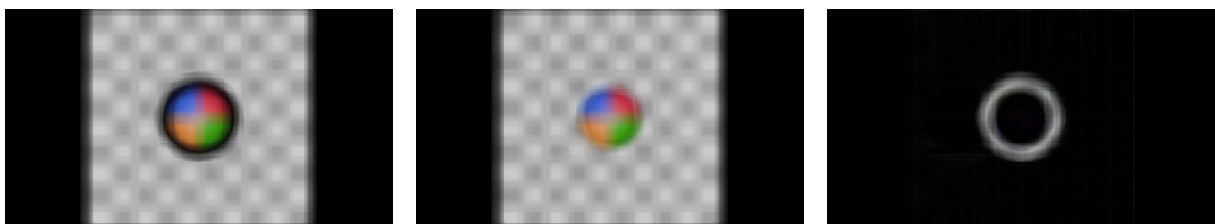


Figure 3.2: On the left the plenoptic output single-view. At the center the plenoptic output multi-view. On the right the difference image.

Compared to the single-view reconstruction, the multi-view result visibly removes dark boundaries around the sphere. These regions correspond to disocclusions generated by missing geometry in the original single-view setup acquisition. The difference image emphasizes this by confirming that the multi-view reconstruction mainly affects regions previously impacted by disocclusions.

We computed as well quantitative metric by identifying recovered pixels: all that pixels that were invalid (black) in the single-view reconstruction and became valid in the multi-view reconstruction.

Figure 3.3 highlights the recovered pixels in green.

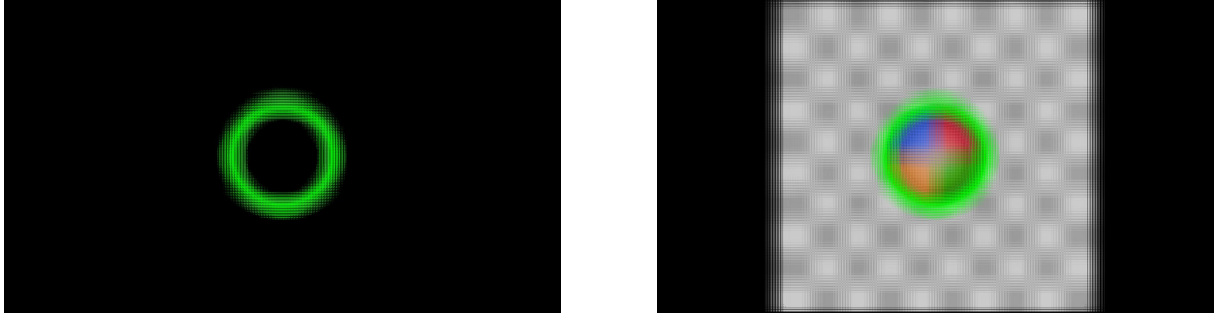


Figure 3.3: On the left the highlighted pixels recovered by the multi-view reconstruction. On the right the overlay visualization of the recovered pixels.

The recovered pixels are mainly concentrated around the silhouette of the sphere, which is consistent with the expected behavior of multi-view geometry compensation. Recovery ration is computed through the following equation:

$$RecoveryRatio = \frac{RecoveredPixels}{InvalidPixelsSingleView} \quad (3.4)$$

Moreover, the density of the final plenoptic image has been taken into account. This metric measures how much valid information is contained by the plenoptic image.

$$Density = 1 - \frac{invalidpixels}{totalpixels} \quad (3.5)$$

The quatitative metrics analysis is reported in the table below:

Table 3.2: Multi-View Validation Metrics

Metric	Value
Recovered Pixels	350,023
Recovery Ratio	5.16%
Single-View Valid Pixel Density	53.98%
Multi-View Valid Pixel Density	56.23%

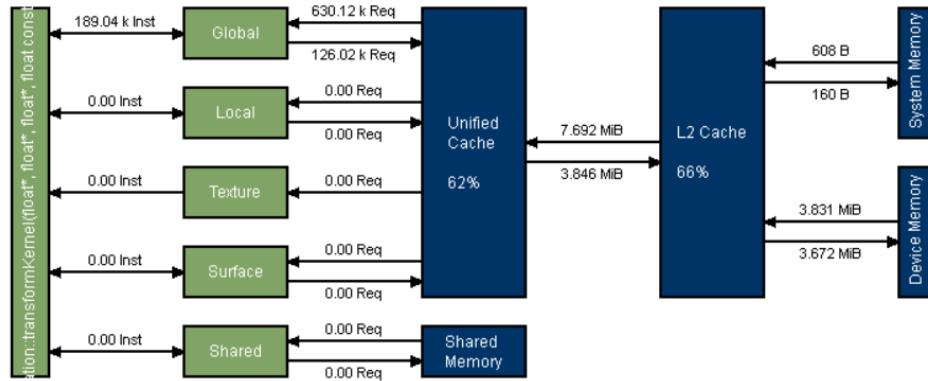
The multi-view pipeline recovers roughly 5% of the lost pixels by the single-view pipeline. We need to keep in mind that we had only one dataset and this could limit the validation. In particular, this value of recovery ratio is tightly connected to the fact that most part of the invalid pixels are background one. Regarding the density, we observe how multi-view increases the quantity of useful information, extending the geometry covering.

Variable	Achieved	Theoretical	Device Limit	Grid Size: [1313,1,1] (1313 blocks)Block Size: [256,1,1]
Occupancy Per SM				
Active Blocks		8	32	
Active Warps	55.49	64	64	
Active Threads		2048	2048	
Occupancy	86.7%	100%	100%	
Warp				
Threads/Block		256	1024	
Warps/Block		8	32	
Block Limit		8	32	
Registers				
Registers/Thread		26	255	
Registers/Block		8192	65536	
Block Limit		8	32	
Shared Memory				
Shared Memory/Block		0	98304	
Block Limit		0	32	

(a) Kernel latency



(b) Stalls analysis



(c) Memory Analysis

Figure 3.4: Profiling of the rigid body transformation kernel in the multi-view registration stage.

Profiling results show that the multiview registration kernel achieves a high occupancy of 86.7%. Memory hierarchy analysis reveals a cache utilization of 62% for the Unified Cache and 66% for L2 Cache, with only a few megabytes transferred between cache levels and device memory. However, warp stall analysis identifies memory dependency as the dominant stall reason and this indicates that execution is mostly connected with memory latency delays access rather than by computational bottleneck. Such behavior is consistent with the kernel's operations, where each thread performs a rigid transformation on a point of the input point clouds by applying a rotation and translation before storing the transformed coordinates.

The project presented the redesign of an existing CPU-based plenoptic image generator pipeline into a GPU architecture and its subsequent extension toward a multi-view reconstruction.

About the first contribution, experimental results proved the correctness of the GPU pipeline, achieving a speedup factor of 5.8x. Further improvements in this factor may be observed when evaluating the approach on different datasets or higher-resolution images. Profiling analysis revealed that the implemented kernels achieve high occupancy levels and are mainly limited by memory accesses rather than arithmetic throughput, indicating that the proposed parallelization strategy effectively exploits the available GPU resource.

The second contribution successfully reduced disocclusion artifacts by exploiting multiple viewpoints from multiple cameras. In addition to a clear visual analysis, we performed a quantitative evaluation using metrics to measure the density of valid reconstructed pixels and the recovery of missing geometry compared to the single-view pipeline. When applied to more complex datasets, these metrics could show further improvement. The GPU implementation also preserves a modular and scalable architecture suitable for future extensions.

Bibliography

- [Feh04] Fehn, Christoph (2004). “Depth-image-based rendering (DIBR), compression, and transmission for a new approach on 3D-TV”. In: *Proceedings of SPIE* 5291, pp. 93–104. DOI: [10.1117/12.526937](https://doi.org/10.1117/12.526937).
- [Fer+26] Ferreira, Brenno et al. (2026). “Large-Size Integral Imaging Display with Depth Image-Based Plenoptic Rendering”. In: *SPIE*.
- [Gen13] Geng, Jason (2013). “Three-dimensional display technologies”. In: *Advances in Optics and Photonics* 5.4, pp. 456–535. DOI: [10.1364/AOP.5.000456](https://doi.org/10.1364/AOP.5.000456).
- [Hof+08] Hoffman, David M. et al. (2008). “Vergence–accommodation conflicts hinder visual performance and cause visual fatigue”. In: *Journal of Vision* 8.3, pp. 33–33. DOI: [10.1167/8.3.33](https://doi.org/10.1167/8.3.33). URL: <https://jov.arvojournals.org/article.aspx?articleid=2122611>.
- [Kle01] Kleywegt, Gerard J. (2001). “Rotations and rotation matrices”. In: *Acta Crystallographica Section D* 57.10, pp. 1455–1462. URL: <https://journals.iucr.org/d/issues/2001/10/00/ba5006/>.
- [Lip08] Lippmann, Gabriel (1908). “Épreuves réversibles donnant la sensation du relief”. In: *Journal de Physique Théorique et Appliquée* 7.1, pp. 821–825. DOI: [10.1051/jphystap:019080070082100](https://doi.org/10.1051/jphystap:019080070082100).
- [MJ18] Martínez-Corral, Manuel and Bahram Javidi (2018). “Fundamentals of 3D imaging and displays: a tutorial on integral imaging, light-field, and plenoptic systems”. In: *Advances in Optics and Photonics* 10.3, pp. 512–566. DOI: [10.1364/AOP.10.000512](https://doi.org/10.1364/AOP.10.000512).
- [Mud+12] Muddala, S. et al. (2012). “Efficient hole filling strategy for depth image based rendering view synthesis”. In: *International Journal of Computer Applications* 41.15, pp. 34–39.
- [Ziv26] Zivid (2026). *Position, Orientation and Coordinate Transformations*. URL: <https://support.zivid.com/en/latest/camera/reference-articles/position-orientation-coordinate-transform.html>.